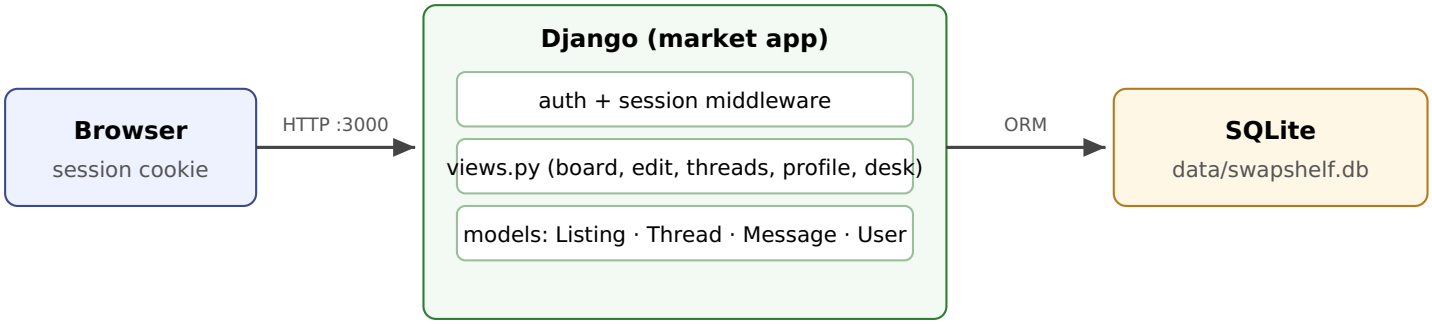
SwapShelf — Security Writeup

Case Study 5 · CPEG 470 · Safowan Raiyean

Architecture sketch



Walking the running app answers the four prompts:

- **How a listing id becomes a row:** `/listings/<int:pk>/` routes to `listing_detail`, which calls `get_object_or_404(Listing, pk=pk)` — the id is the primary key looked up in the `Listing` table.
- **Pages a logged-out visitor can open:** the board (`/`) with its category filter, any listing detail, register, and sign in. Everything else is behind `@login_required`.
- **What the profile form writes back:** it saves fields on the logged-in `User` row — and, before the patch, `is_staff` along with name and email.
- **What the department desk is for:** `/desk/` shows a hold-closet location and a reimbursement code to `is_staff` accounts only. It is meant for department board staff, set by an admin.

The defect — privilege escalation through the profile form

The "Show my name on the department board" checkbox on `/profile/` is not a cosmetic flag. In `forms.py` it is bound directly to the `User.is_staff` field:

```
class ProfileForm(forms.ModelForm):
    class Meta:
        model = User
        fields = ["first_name", "last_name", "email", "is_staff"]   # <- privilege flag
```

So any signed-in user can grant themselves staff rights just by submitting their own profile. `is_staff` controls two things: access to the `/desk/` page (which leaks the reimbursement code `DESK-2290`) and access to the Django admin site at `/admin/`. This is a classic mass-assignment / broken-access-control bug — a privilege field exposed as if it were a user preference.

How to trigger it

- Register a brand-new account and sign in.
- Open `/desk/` — it says "for department board accounts" (no access).
- Go to `/profile/`, tick **"Show my name on the department board"**, save.
- Reload `/desk/` — it now shows `Reimbursement code DESK-2290`, and `/admin/` is reachable.

Confirmed with curl against the running app: a fresh account reached the admin index (HTTP 200) and the desk leaked the code after one profile submit.

The patch

Drop `is_staff` from the editable form. Staff is assigned by an admin, not self-service.

```
# forms.py  (after)
class ProfileForm(forms.ModelForm):
    class Meta:
        model = User
        fields = ["first_name", "last_name", "email"]
```

Posting normal profile edits still works; a submitted `is_staff` value is simply ignored because the field is no longer bound.

Related access-control fixes

Walking the app turned up two more broken-access-control holes in the same class, so I closed them in the same patch:

Hole	Fix
Any signed-in user could read or post to **any** message thread by id (`/threads/<pk>/`) — the seeded threads contain pickup locker codes.	`thread_detail` now raises `Http404` unless the user is the thread's buyer or the listing's owner.
Any signed-in user could edit, mark sold, or delete **anyone's** listing (`/listings/<pk>/edit|sold|delete/`).	Those views now look up the listing with `owner=request.user`, so a non-owner gets a 404 and the detail page only shows edit/sold controls to the owner.

Testing

Ran the original on :3000 and the patched build on :3100, side by side.

- **Escalation blocked:** a fresh account submitting `is_staff=on` stays `is_staff=False`; `/admin/` redirects to login and `/desk/` stays locked.
- **Thread privacy:** a non-party account gets 404 on `/threads/1/`; the buyer and seller still open it (200).
- **Listing ownership:** a non-owner gets 404 editing listing #1 and no state change (still "available"); the owner edits normally.
- **Normal use intact:** register, sign in, post a listing, edit your own, message a seller, and read your own thread all work.